15
Design YouTube

In this chapter, you are asked to design YouTube. The solution to this question can be applied to other interview questions like designing a video sharing platform such as Netflix and Hulu. Figure 1 shows the layout of this chapter.

YouTube broke simple, content creators upload videos and viewers click play. It is hardly that simple!! Not really. There are lots of complex technologies underneath the hood. Let us look at some impressive statistics, demographics, and fun facts of YouTube in 2020 [1] [2].

- Total number of monthly active users: 2 billion.
- Number of hours watched every day: 1 billion.
- 70% of views are from mobile.
- 50 million creators on YouTube.
- YouTube's revenue was $15 billion for the full year 2019, up 36% from 2018.
- YouTube is the second most popular website (after Google).
- YouTube is available in 80 different languages.

From these statistics, we know YouTube is enormous, global and keeps a lot of traffic.

Step 1 - Understand the problem and establish design scope

As revealed in Figure 1, benefits watching a video, you can click on a video to watch it. Text, comments, comment, share, or like a video, are some of the popular features that are available. In a nutshell, we need to design a video sharing platform, or, in 30-minute interview. This is a very broad question. We need to ask questions to narrow down the scope.

Candidate: What features are important?
Interviewer: It should be possible to upload and watch short videos. The maximum allowed video size is 1GB.
Candidate: What devices do we need to support?
Interviewer: Mobile apps, web browsers, and smart TV.
Candidate: How many daily active users do we have?
Interviewer: 5 million.
Candidate: What is the average daily time spent on the product?
Interviewer: 30 minutes.
Candidate: Do we need to support international users?
Interviewer: Yes, a large percentage of users are international users.
Candidate: What are the supported video resolutions and formats?
Interviewer: The system accepts most of the video resolutions and formats.
Candidate: Are encryption required?
Interviewer: Yes.
Candidate: Any file size requirement for videos?
Interviewer: No, platform focuses on short and medium sized videos. The maximum allowed video size is 1GB.
Candidate: Can we leverage some of the existing cloud infrastructure provided by Amazon, Google, or Microsoft?
Interviewer: That is a great question. Building everything from scratch is unrealistic for most companies, it is recommended to leverage some of the existing cloud services.

Based on the above conversation, we focus our video streaming service with the following features:

- Ability to upload videos to YouTube.
- Smooth video streaming.
- Ability to change video quality.
- Low infrastructure cost.
- High availability, scalability, and reliability requirements.
- Clients supported: mobile apps, web browser, and smart TV.

Back of the envelope estimation

The following estimations are based on many assumptions, so it is important to communicate with the interviewer to make sure she is on the same page.

- Assume the product has 5 million daily active users (DAU).
- Each user watches five videos a day.
- 10% of users upload 1 video per day.
- Assume the average video size is 300 MB.
- A video uploaded would need transfer 3 million * 300 = 900 ~ 1000TB
- CDN cost.
- When cloud CDN serves a video, you are charged for data transferred out of the CDN.
- Let us use CloudFlare CDN (CloudFlare for cost estimation) offer 20 TB of traffic at no cost, served from the United States. The average cost per GB is $0.02. For simplicity, we only calculate the cost of video streaming.
- Cost for CDN per day: 20,000 * 0.02 = $400
- Assume 10% of traffic requires streaming video streaming service with the following features:
- Assume the average daily time spent on the product is 30 minutes.
- Total number of hours watched by users: 5 million * 30 / 60 = 2500 hours
- Assume 10% users watch videos in a different resolution, 720p for example. The average bandwidth needed for 720p is 3 Mbps.
- Total storage space needed: 2 million * 10% * 60 MB / 1024 = 1200GB
- Cost of storage: Let us use Amazon S3 standard storage as our storage solution. The first 50 terabytes of storage are free, the next 450 terabytes are $0.023 per GB. Assuming we only need 1200GB of storage, the cost is: (1200 - 50) * 0.023 = $26.45

Service	Cost
CDN	$400
Storage	$26.45
Total	$426.45

Figure 2: Cost estimation

Step 2 - Propose high-level design and get buy-in

In the back of envelope, the important observations including storage, cost, and bulk nature of traffic. Building from scratch is unrealistic for most of the companies, we focus on how to leverage cloud services. Some location might also play a role building everything by ourselves. Reasons are listed below:

- System design interviews are not about building everything from scratch. Within the limited time frame, stressing the right technology to be a job right is more important than explaining how the technology works in detail. For instance, mentioning video transcoding architecture is important, but we don't need to spend the entire interview discussing how to build our own video codec.
- Building scalable video storage or CDN is extremely complex and costly. Even large companies like Netflix or Facebook do not build everything themselves. Netflix leverages Amazon's cloud services [3], and Facebook uses Akamai's CDN [4].

At the high level, the system comprises three components (Figure 3).

Figure 3

Client: You can watch YouTube on your computer, mobile phone, and smart TV.

CDN: Videos are stored in CDN. When you press play, a video is streamed from the CDN.

API servers: Everything else except video streaming goes through API servers. This includes feed recommendation, generating video upload URL, updating metadata database and cache, user signup, etc.

In the questionnaire session, the interviewer showed interests in two flows:

- Video uploading flow
- Video streaming flow

We will evolve the high-level design for each of them.

Video uploading flow

Figure 4 shows the high-level design for the video uploading.

Figure 4

Components of the following are created:

- User: A user watches YouTube on devices such as a computer, mobile phone, or smart TV.
- Load balancer: A load balancer evenly distributes requests among API servers.
- API servers: All user requests go through API servers except video streaming.
- Metadata database: Video metadata are stored in Metadata DB. It is sharded and replicated to meet performance and high availability requirements.
- Metadata cache: For better performance, video metadata and user objects are cached.
- Original storage: A blob storage system is used to store original video files. It provides lots of features to integrate with other cloud services.
- Transcoding servers: Video transcoding servers are used to transcode videos. It is a time-consuming and computation-intensive process. When a video is uploaded, the transcoding server needs to transcode a video to different resolutions and formats.
- Transcoding queue: A queue is used to store video transcoding tasks.
- Transcoding completion queue: A queue is used to store information about completed transcoding tasks. The queue contains the following information: a new video file has been transcoded, and its storage location.
- Completion handler: This consists of a list of servers that pull jobs from the completion queue and update metadata cache and database.

Note that we underscored each component individually, let us examine how the video uploading flow works. The flow is broken down into two processes running in parallel:

- Upload the actual video.
- Update video metadata. Metadata contains information about video URL, size, resolution, format, user, etc.

Flow a: upload the actual video

Figure 5

Figure 5 shows how to upload an mp4 video file. The explanation is shown below:

1. Video are uploaded to the original storage.
2. Transcoding servers fetch videos from the original storage and start transcoding.
3. Transcoding servers notify API servers that the video is transcoded.
4. API servers inform the client that the video is successfully uploaded and is ready for streaming.
5. The metadata database and cache are updated to the completion queue.
   5a. Transcoded videos are distributed to CDN.
   5b. Transcoding completion events are queued in the completion queue.
   5c. Completion handler contains a job that continuously pull event data from the queue.
   5d. It uses the event data to update metadata database and cache when video transcoding is complete.
6. API servers inform the client that the video is successfully uploaded and is ready for streaming.

Flow b: update the metadata

While a video is being uploaded to original storage, the client in parallel sends a request to update the video metadata as shown in Figure 6. The request contains video metadata, including file name, size, format, etc. API servers update the metadata cache and database.

Figure 6

Video streaming flow

When a video is ready for streaming, it could be viewed on a device such as a phone, TV, or computer. Different protocols are used for different devices. For instance, some protocols support only iOS devices, and others may work well with Android. Most of the protocols are based on HTTP, a protocol that has been proven to work well over the Internet. We will only focus on the following streaming protocols for this discussion:

- MPEG-DASH. MPEG stands for "Moving Pictures Experts Group" and DASH stands for "Dynamic Adaptive Streaming over HTTP".
- Apple HLS. HLS stands for "HTTP Live Streaming".
- Microsoft Smooth Streaming.
- Adobe HTTP Dynamic Streaming (HDS).

You do not need to fully understand the following streaming protocols before as they are non-trivial details that require specific knowledge and experience. Nevertheless, it is good to know that there are many different protocols available because there is no standard protocol adopted by all devices. When we design a video streaming service, we need to choose the right streaming protocol to support our use cases. To learn more about streaming protocols, have a an unpaired article [5].

The following protocols are used for video streaming in our design, which we choose as you will deliver the video. Thus, there is very little latency. Figure 7 shows a high level design for video streaming.

Figure 7

Step 3 - Design deep dive

In the high level design, the storing system is broken down to two parts: video uploading flow and video streaming flow. In this section, we will refine both flows with important optimizations and introduce error handling mechanisms.

Video transcoding

When you record a video, the device usually it records a raw video format. Raw video consumes a lot of storage. If you send the video in the raw format uncoded, you waste a lot of bandwidth. Raw video must be encoded into other formats for storage and for streaming. Most of the raw video files are unprocessed over time. A higher bitrate generally means higher video quality. High bitrate streams need more processing power and fast Internet speed.

Video transcoding is important for the following reasons:

- Raw video consumes huge storage space.
- Many devices and browsers only support certain types of video formats. Thus, it is important to encode a video to different formats for compatibility.
- To ensure users watch high quality videos while maintaining smooth streaming, it is a good idea to deliver higher resolution video to users who have high network bandwidth and lower resolution video to users who have low bandwidth.
- Network conditions change, especially on mobile devices. It is wise to switch to lower quality streams, switching video quality between frames if network bandwidth is insufficient for higher quality streaming.

Many types of encoding formats are available, however, most of them contain two parts:

- Container: This is like a basket that contains files that follow the traits, and extensions. You can tell the container format by the file extension, such as .avi, .mov, or .mp4.
- Codec: These are compression and decompression algorithms to reduce the video size while preserving the video quality. The most commonly used video codecs are H.264, VP9, and HEVC.

Directed acyclic graph (DAG) model

Transcoding a video is computationally expensive and time-consuming. Besides, different content creators may have different video processing requirements. For instance, some content creators may add watermarks on top of their videos, some provide thumbnail images for their videos, and most of videos need to be encoded in various resolutions, codecs, bitrates, etc.

To support different video processing pipelines and maintain high parallelism, it is important to add some level of abstraction and let client programmers define what tasks need to execute. For example, Facebook recently introduced streaming video engine, a new framework for building flexible and scalable video processing pipelines. It uses a directed acyclic graph (DAG) programming model, which defines tasks in stages so they can be executed sequentially or concurrently [6]. In our design, we adopt a similar DAG model to achieve flexibility and parallelism. Figure 8 represents a DAG for video transcoding.

Figure 8

In Figure 8, the original video is split into video, audio, and metadata. Many other operations can run on the tasks that can be applied on a video file.

- Inspection: Make sure video files are good quality and are not malformed.
- Video encodings: Videos are encoded to support different resolutions, codecs, bitrates, etc. Figure 9 shows an example of video encoded files.
- Thumbnail: Thumbnails can either be uploaded by a user or automatically generated by the system.
- Watermark: An image overlay on top of your video contains identifying information about your video.

Figure 9

Video transcoding architecture

The proposed video transcoding architecture that leverages the cloud services, is shown in Figure 10.

Figure 10

The architecture has six main components: preprocessor, DAG scheduler, resource manager, task workers, temporary storage, and encoded video are the output. Let us take a closer look at each component.

Preprocessor

Figure 11

The preprocessor has 4 responsibilities:

1. Video splitting. Video stream is split or further split into smaller Group of Pictures (GOP) alignment. GOP is a group/sequence of frames arranged in a specific order. It contains no or independently playable unit, usually a few seconds in length.
2. Some old mobile devices or browsers might not support the video segment, the preprocessor will split the video to GOP alignment for old clients.
3. DAG generation. The preprocessor generates DAG based on configuration files written programmers write. Figure 12 is a simplified DAG representation which has 2 nodes and 1 edge:

Figure 12

This DAG representation is generated from the two configuration files below (Figure 13):

```
[ video.name = "cat.mp4"
  video.source = "s3://..."
  stream.type = "video"
  stream.codec = "h264"
  stream.bitrate = [2, 4, 8]
  stream.height = [480, 720, 1080]
  stream.width = [853, 1280, 1920]
  stream.output = "tmp/mp4/..."
]
```

```
[ video.name = "cat.mp4"
  video.source = "s3://..."
  stream.type = "video"
  stream.codec = "hevc"
  stream.bitrate = [1, 3, 5]
  stream.height = [480, 720, 1080]
  stream.width = [853, 1280, 1920]
  stream.output = "tmp/hevc/..."
]
```

Figure 13 (Figure 20)

4. Cache data. The preprocessor is a cache for segmented video. For better reliability, the preprocessor stores GOPs and metadata in temporary storage. If video encoding fails, the preprocessor could pick up from the last segment.

DAG scheduler

Figure 14

The DAG scheduler splits a DAG graph into stages of tasks and puts them in the task queue in the resource manager. Figure 15 shows an example of how the DAG scheduler works.

Figure 15

As shown in Figure 15, the original video is split into three stages: Stage 1: video, audio, and metadata. The video file is further split into two tasks in stage 2: video encoding and thumbnail. The audio file requires audio encoding as part of the stage 2 tasks.

Resource manager

Figure 16

The resource manager manages as follows:

- The task queue: It is a priority queue that contains tasks to be executed.
- Running queue: It is a priority queue that contains worker utilization info.
- Running queue: It contains info about the currently running tasks and workers running the tasks.
- Task scheduler: It picks the optimal task/worker, and instructs the chosen task worker to execute the job.

Task scheduler
Task queue
Running queue
Task worker
Task worker
Task worker

Figure 18

Task workers run the tasks which are defined in the DAG. Different task workers may run different tasks as shown in Figure 19.

Figure 19

Temporary storage

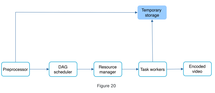

Figure 20

Multiple storage systems are used here. The choice of storage system depends on factors like data type, data size, access frequency, data life span, etc. For instance, metadata is frequently accessed by workers, and the data size is usually small. Thus, caching metadata in memory is a good idea. For video or audio data, we put them in blob storage. Data in temporary storage is freed up once the corresponding video processing is complete.

Encoded video

Figure 21

Encoded video is the final output of the encoding pipeline. Here is an example of the output: funny_720p.mp4.

System optimizations

At this point, you ought to have good understanding about the video uploading flow, video streaming flow and video transcoding. Next, we will refine the system with optimizations, including speed, safety, and cost-saving.

Speed optimization: parallelize video uploading

Uploading a video as a whole unit is inefficient. We can split a video into smaller chunks by GOP alignment as shown in Figure 22.

Figure 22

This allows fast resumable uploads when the previous upload failed. The job of splitting a video file by GOP can be implemented by the client to improve the upload speed as shown in Figure 23.

Figure 23

Speed optimization: place upload centers close to users

Another way to improve the upload speed is by setting up multiple upload centers across the globe (Figure 24). People in the United States can upload videos to the North America upload center, and people in China can upload videos to the Asian upload center. To achieve this, we use CDN as upload centers.

Figure 24

Speed optimization: parallelism everywhere

Achieving low latency requires serious efforts. Another optimization is to build a loosely coupled system and enable high parallelism.

Our design needs some modifications to achieve high parallelism. Let us zoom in to the flow of how a video is transferred from original storage to the CDN. The flow is shown in Figure 25, revealing that the output depends on the input of the previous step. This dependency makes parallelism difficult.

Figure 25

To make the system more loosely coupled, we introduced message queues as shown in Figure 26. Let us use an example to explain how message queues make the system more loosely coupled.

- Before the message queue is introduced, the encoding module must wait for the output of the download module.
- After the message queue is introduced, the encoding module does not need to wait for the output of the download module anymore. If there are events in the message queue, the encoding module can execute those jobs in parallel.

Figure 26

Safety optimization: pre-signed upload URL

Safety is one of the most important aspects of any product. To ensure only authorized users upload videos to the right location, we introduce pre-signed URLs as shown in Figure 27.

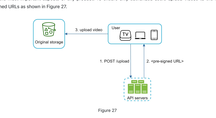

Figure 27

The upload flow is updated as follows:

1. The client makes a HTTP request to API servers to fetch the pre-signed URL, which gives the access permission to the object identified in the URL. The term pre-signed URL is used by companies offering files to Amazon S3. Other cloud service providers might use a different name. For instance, Microsoft Azure blob storage supports the same feature, but call it "Shared Access Signature" [10].

2. API servers respond with a pre-signed URL.

3. Once the client receives the response, it uploads the video using the pre-signed URL.

Safety optimization: protect your videos

Many content makers are reluctant to post videos online because they fear their original videos will be stolen. To protect copyrighted videos, we can adopt one of the following three safety options:

- Digital rights management (DRM) systems: Three major DRM systems are Apple FairPlay, Google Widevine, and Microsoft PlayReady.
- AES encryption: You can encrypt a video and configure an authorization policy. The encrypted video will be decrypted upon playback. This ensures that only authorized users can watch an encrypted video.
- Visual watermarking: This is an image overlay on top of your video that contains identifying information for your video. It can be your company logo or company name.

Cost-saving optimization

CDN is a crucial component of our system. It ensures fast video delivery on a global scale. However, from the back of the envelope calculation, we know CDN is expensive, especially when the data size is large. How can we reduce the cost?

Previous research shows that YouTube video streams follow long-tail distribution [11] [12]. It means a few popular videos are accessed frequently but many others have few or no viewers. Based on this observation, we implement a few optimizations:

1. Only serve the *most popular* videos from CDN and other videos from our high capacity storage video servers (Figure 28).

Figure 28

2. For less popular content, we may not need to store many encoded video versions. Short videos can be encoded on-demand.

3. Some videos are popular only in certain regions. There is no need to distribute these videos to other regions.

4. Build your own CDN like Netflix and partner with Internet Service Providers (ISPs). Building your CDN is a giant project; however, this could make sense for large streaming companies. An ISP can be Comcast, AT&T, Verizon, or other internet providers. ISPs are located all around the world and are close to users. By partnering with ISPs, you can improve the viewing experience and reduce the bandwidth charges.

All those optimizations are based on content popularity, user access pattern, video size, etc. It is important to analyze historical viewing patterns before doing any optimization. Here are some of the interesting articles on this topic: [10] [13].

Error handling

For a large-scale system, system errors are unavoidable. To build a highly fault-tolerant system, we must handle errors gracefully and recover from them fast. Two types of errors exist:

- Recoverable error. For recoverable errors such as video segment fails to transcode, the general idea is to retry the operation a few times. If the task continues to fail and the system believes it is not recoverable, it returns a proper error code to the client.
- Non-recoverable error. For non-recoverable errors such as malformed video format, the system stops the running tasks associated with the video and returns the proper error code to the client.

Typical errors for each system component are covered by the following playbook:

- Upload error: retry a few times.
- Split video error: if older versions of clients cannot split videos by GOP alignment, the entire video is passed to the server. The job of splitting video is done on the server-side.
- Transcoding error: retry.
- Preprocessor error: regenerate DAG diagram.
- DAG scheduler error: reschedule a task.
- Resource manager queue down: use a replica.
- Task worker down: retry the task on a new worker.
- API server down: API servers are stateless so requests will be directed to a different API server.
- Metadata cache server down: data is replicated multiple times. If one node goes down, you can still access other nodes to fetch data. We can bring up a new cache server to replace the dead one.
- Metadata DB server down:
  - Master is down. If the master is down, promote one of the slaves to act as the new master.
  - Slave is down. If a slave goes down, you can use another slave for reads and bring up another database server to replace the dead one.

Step 4 - Wrap up

In this chapter, we presented the architecture design for video streaming services like YouTube. If there is extra time at the end of the interview, here are a few additional points:

- Scale the API tier: Because API servers are stateless, it is easy to scale API tier horizontally.
- Scale the database: You can talk about database replication and sharding.
- Live streaming: It refers to the process of how a video is recorded and broadcasted in real time. Although our system is not designed specifically for live streaming, live streaming and non-live streaming have some similarities: both require uploading, encoding, and streaming. The notable differences are:
  - Live streaming has a higher latency requirement, so it might need a different streaming protocol.
  - Live streaming has a lower requirement for parallelism because small chunks of data are already processed in real-time.
  - Live streaming requires different sets of error handling. Any error handling that takes too much time is not acceptable.
- Video takedowns: Videos that violate copyright, child pornography, or other illegal acts shall be removed. Some can be discovered by the system during the upload process, while others might be discovered through user flagging.

Congratulations on getting this far! Now give yourself a pat on the back. Good job!

Reference materials

[1] YouTube by the numbers:
https://www.omnicoreagency.com/youtube-statistics/

[2] 2019 YouTube Demographics:

https://blog.hubspot.com/marketing/youtube-demographics

[3] CloudFront Pricing:
https://aws.amazon.com/cloudfront/pricing/

[4] Netflix on AWS: https://aws.amazon.com/solutions/case-studies/netflix/

[5] Akamai homepage: https://www.akamai.com/

[6] Binary large object:
https://en.wikipedia.org/wiki/Binary_large_object

[7] Here's What You Need to Know About Streaming Protocols:
https://www.dacast.com/blog/streaming-protocols/

[8] SVE: Distributed Video Processing at Facebook Scale:
https://www.cs.princeton.edu/~wlloyd/papers/sve-sosp17.pdf

[9] Weibo video processing architecture (in Chinese):
https://www.upyun.com/opentalk/399.html

[10] Delegate access with a shared access signature:
https://docs.microsoft.com/en-us/rest/api/storageservices/delegate-access-with-shared-access-signature

[11] YouTube scalability talk by early YouTube employee:
https://www.youtube.com/watch?v=w7i4HYxub2wY9

[12] Understanding the characteristics of internet short video sharing: A youtube-based measurement study.
https://arxiv.org/pdf/2107.14125.pdf

[13] Content Popularity for Open Connect:
https://netflixtechblog.com/content-popularity-for-open-connect-b86d56f613b

Mark as Complete

Partner With Us
Be an Affiliate
Become a Contributor

Company & Legal
Our Team
Newsletter
Privacy Policy
Terms of Service
Do Not Sell or Share My Personal Information

Support
hi@bytebytego.com
Report a Bug

Resources
YouTube
Visual Dev Guides
Find Jobs
Prepare for Coding Interviews

Ask Alex

Copyright ©2022-2026 BytebyteGo Inc. All rights reserved.